

Object-oriented Programming 2 KU

OSM, JTS, Geometries, and Data Loading

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)

What is Open Street Maps (OSM)?

A short introduction into GIS's (and OSM)

- GIS is a Geographic information system
- OSM has the following types:
 - 1D: Points: (OSM: Nodes)
 - 2D: Lines - Polylines - Polygon: (OSM: Ways)
 - 2D: Multiple Polygon (holes) - Multiple Geometries: (OSM: Relations)

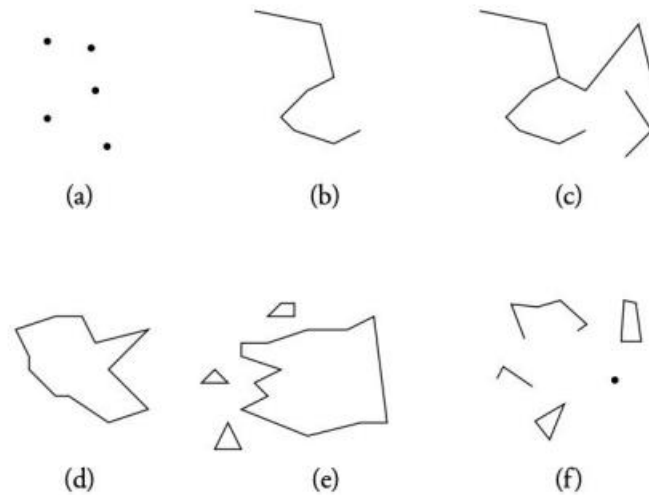


Figure 3.5 Candidates for geometric types: points (a), polyline (b), complex polyline (c), polygon (d), polygon set (e), and mixed (f).

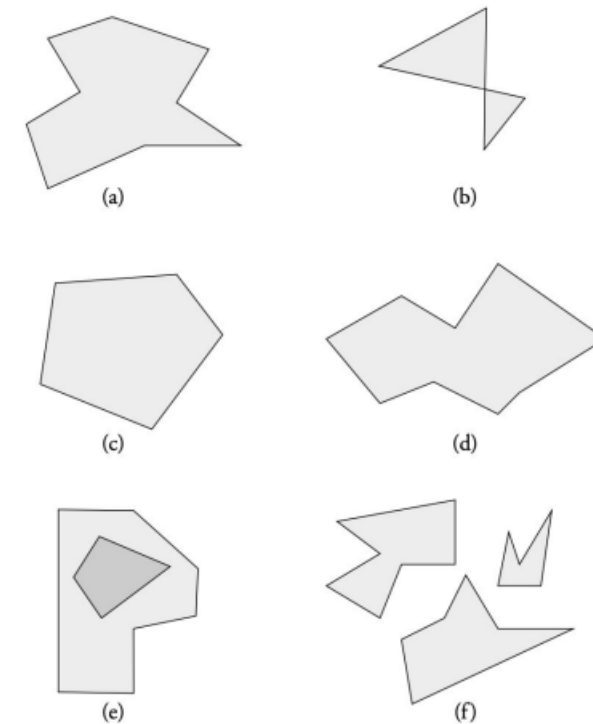
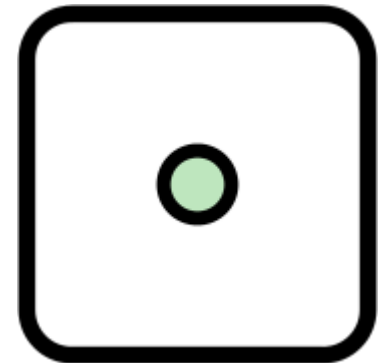


Figure 2.2 Examples of 2D objects: simple polygon (a), non-simple polygon (b), convex polygon (c), monotone polygon (d), polygon with hole (e), and region (f).

A short introduction into GIS's (and OSM)



- OSM has the following [types](#):
 - 1D: Points: (OSM: [Nodes](#))
 - Consists of a single point in space defined by its **latitude**, **longitude** and **node id**.
 - Nodes can be used to define standalone point features but are more often used to define the shape or "path" of a [way](#).
 - As of March 2024, OpenStreetMap contains over [9 billion nodes](#).



A short introduction into GIS's (and OSM)

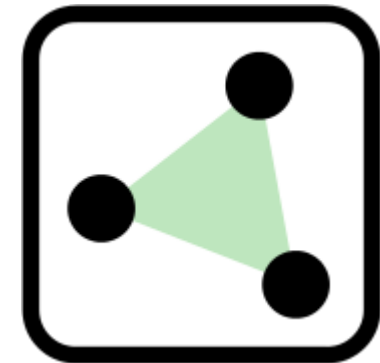


- OSM has the following [types](#):
 - 2D: Lines - Polylines - Polygon: (OSM: [Ways](#))
 - In everyday language, it is a line.
 - A way typically represents a linear feature on the ground (such as a road, wall, or river).
 - Technically, a way is an ordered list of [nodes](#).
 - It usually also has at least one [tag](#) or participates in a [relation](#).
 - A way can have between 2 and 2,000 nodes, although it is possible that faulty ways with zero or one node exist.
 - A way can be [open](#) or [closed](#).

A short introduction into GIS's (and OSM)



- OSM has the following [types](#):
 - 2D: Lines - Polylines - Polygon: (OSM: [Ways](#))
 - A way can be [open](#) or [closed](#).
 - Closed Ways:
 - Known as **closed polylines**
 - In a *closed way* the last node of the way is identical with the first node. A closed way may be interpreted either as a closed polyline (a linear representation of a feature), or as an [area](#), or both, depending on its tags and the tags on its [relations](#).



A short introduction into GIS's (and OSM)



- OSM has the following [types](#):
 - 2D: Lines - Polylines - Polygon: (OSM: [Ways](#))
 - A way can be [open](#) or [closed](#).
 - Open Ways:
 - Known as **open polylines**
 - In an *open way* (a linear representation of a [feature](#)), the first and last nodes are not identical. Common examples of linear representation with open ways include most roads, streams, and railway lines because these start in one place and finish in another.
 - In the database, a way always has a direction, determined by the order of nodes. This is true even if the ground feature it represents is 'two-way' (e.g., most roads, where traffic passes in both directions) or has no direction (e.g., a wall).
 - See here for [how to identify the direction of a 'way'](#).



A short introduction into GIS's (and OSM)



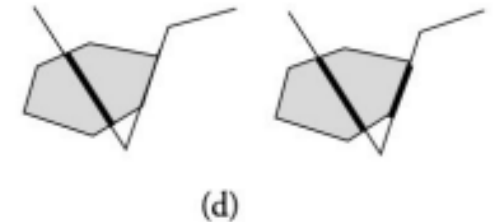
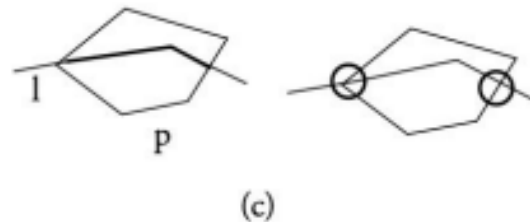
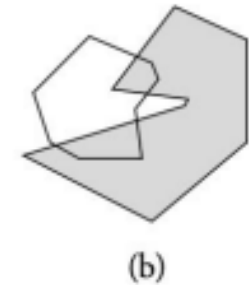
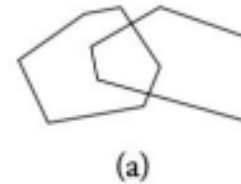
- OSM has the following [types](#):
 - 2D: Multiple Polygon (holes) - Multiple Geometries: (OSM: [Relations](#))
 - **Relations** are structured collections of objects – nodes, ways and other relations.
 - A relation element must have at least the type=* tag, and a group of members which is an ordered list of one or more nodes, ways and/or relations.
 - It is used to define logical or geographic relationships between these different objects (for example, a lake and its island or several roads forming a bus route).
 - A member of a relation can optionally have a role, describing the function that it plays within a relation.
 - Tags describing the collection as a whole should go only on the relation (e.g. `wikidata=*`, `destination=*`, ...), except for the name, if any, which should also go in the relation members.



A short introduction into GIS's (and OSM)

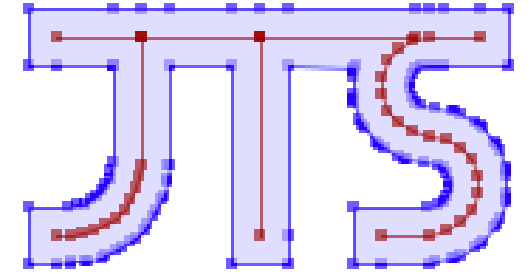


- Operations on the Data types
 - Overlap/intersects whether two entities intersect
 - Intersection: the resulting union of two (or more) geometries
 - Distance to the (minimal) distance to another geometry (depends on the definition!)
 - Area/Length
 - ...

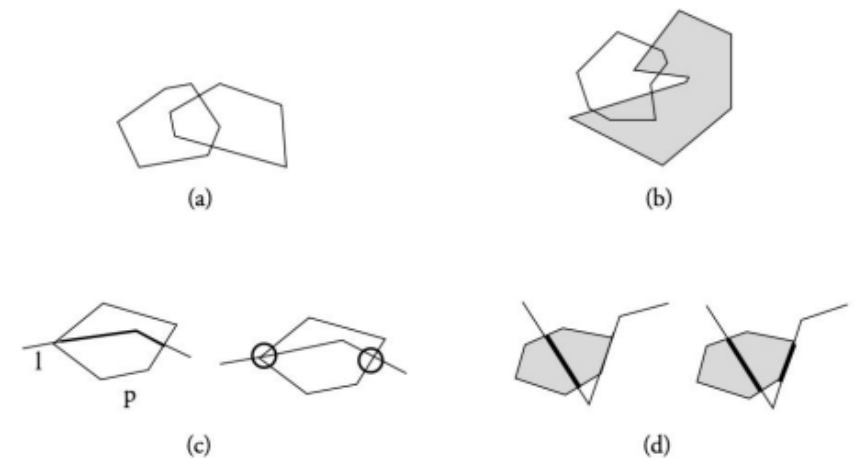


What is JTS?

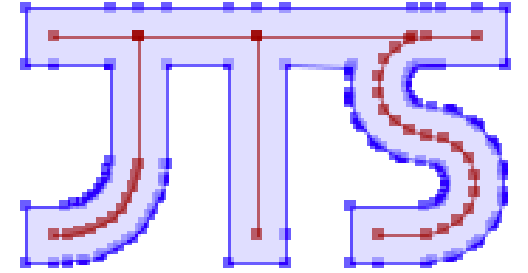
Introduction to JTS



- JTS (Java Topology Suite) is a Java library for creating and manipulating vector geometry.
- It provides a set of tools to handle 2D geometric operations such as intersections, unions, and buffering.
- It is widely used in GIS (Geographic Information Systems) applications.

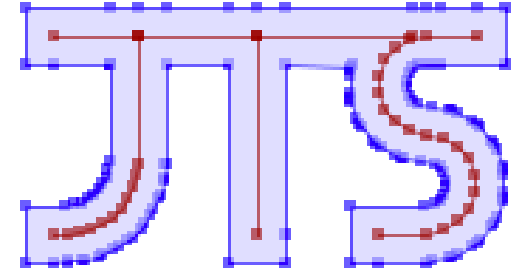


Core Concepts of JTS



- Geometry Class:
 - The base class for all geometric shapes (e.g., Point, LineString, Polygon).
- GeometryFactory:
 - Creates geometry instances (e.g., a polygon or a line).
- Spatial Operations:
 - JTS provides methods like intersects(), intersection(), buffer(), etc.

GeoJSON Format and JTS

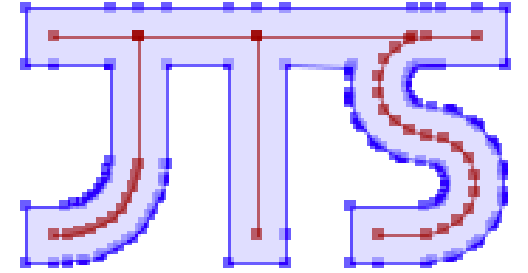


- What is GeoJSON?
 - A format for encoding a variety of geographic data structures.
 - Representations for Point, LineString, Polygon, etc.
- How JTS interacts with GeoJSON:
 - JTS supports GeoJSON input/output, allowing easy integration with web mapping tools and APIs.



Example: JTS and Geometries in Java

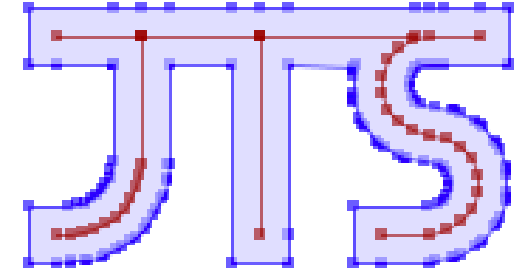
JTS and Geometry in Java



- Follow along if you want again 😊
 - Download and open this project [here from TC](#).
 - The provided example shows an *excellent* introduction to JTS
 - Last years recording [here](#)



Step 1: Reading GeoJSON Data

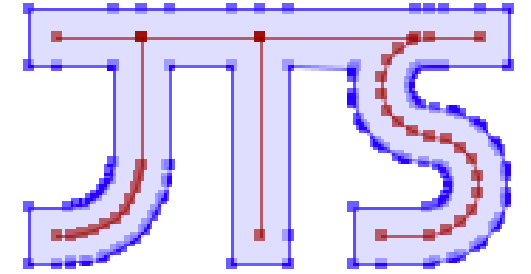


- GeoJsonReader reads GeoJSON format data, turning it into JTS Geometry objects.
- The readResource() method loads the JSON from resources.

```
public static void main(String[] args) {  
    var factory = new GeometryFactory();  
    var geojson_reader = new GeoJsonReader();  
    try {  
        var geo1 = geojson_reader.read(readResource("geo1.json"));  
        var geo2 = geojson_reader.read(readResource("geo2.json"));  
        // Take a look at https://geojson.io/#map=14.7/47.06254/15.45552  
        ...  
    }  
}
```



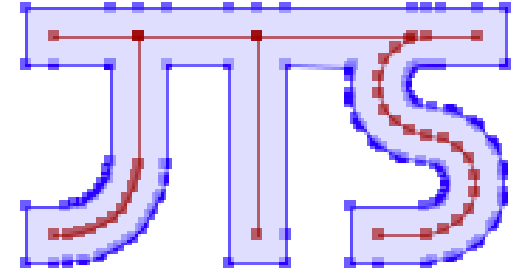
Step 2: Checking for Spatial Relations



- Intersect Operation: Checks whether the two geometries (geo1 and geo2) intersect.
- Intersection Operation: Returns the geometry of their intersection.

```
logger.info("Geo1 intersects Geo2: " + geo1.intersects(geo2));  
logger.info("Geo1 intersection with Geo2: " + geo1.intersection(geo2));
```

Step 3: Working with LineStrings



- The code checks if geo2, which is a LineString is closed
- Converts the LineString to a Polygon

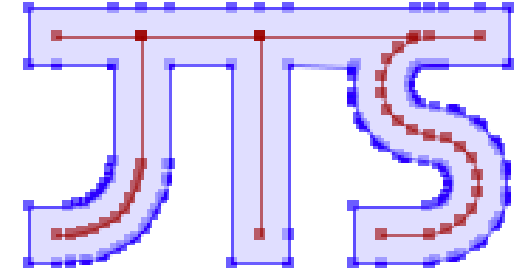
```
GeoJsonWriter writer = new GeoJsonWriter();

logger.info(writer.write(geo2));
logger.info("Geo2 closed: " + ((LineString) geo2).isClosed());

var polygon = factory.createPolygon(geo2.getCoordinates());

logger.info("Polygon from Linestring: " + writer.write(polygon));
```

Step 4: Geometry Collections

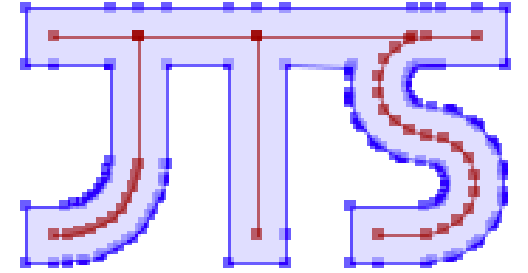


- Creates a collection of geometries (geo1 and the polygon) and logs them.



```
var collection = factory.createGeometryCollection(new Geometry[]{geo1, polygon});  
logger.info("Combined geometries: " + writer.write(collection));
```

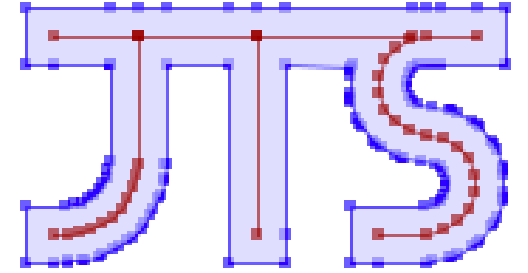
Step 5: Get Length of LineString



```
logger.info("Length of line:" + ((LineString)geo2).getLength());
```



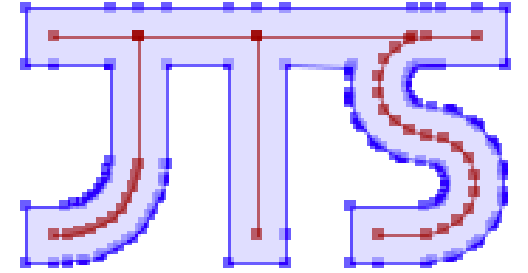
Step 5: Get Length of LineString



```
//DO NOT DO THIS!  
  
logger.info("Length of line:" + ((LineString)geo2).getLength());  
  
// You have to transform first!
```

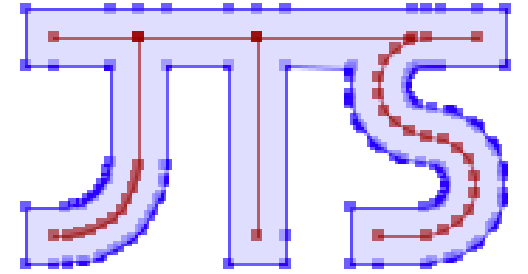


Actual Step 5: Transformation



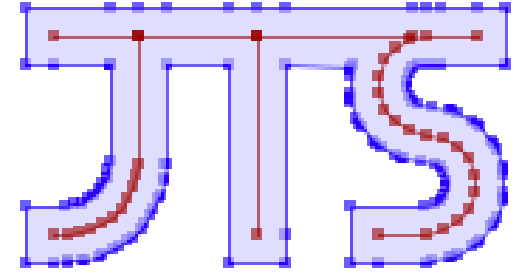
- The coordinates in systems like WGS84 represent angular distances (degrees of latitude and longitude), not direct physical distances (like meters or kilometers).
- You should not calculate geometric properties (like length) on a LineString using latitude and longitude, because those values represent angular degrees, not physical distances.
- Directly calculating the length of a LineString in this coordinate system will give you a result in degrees, which is not meaningful for most spatial applications that need distances in meters or kilometers.
- To get meaningful results (e.g., meters or kilometers), you must transform the coordinates into a projected coordinate system using a tool like Proj4J.

Step 5: Transformation of Coordinates



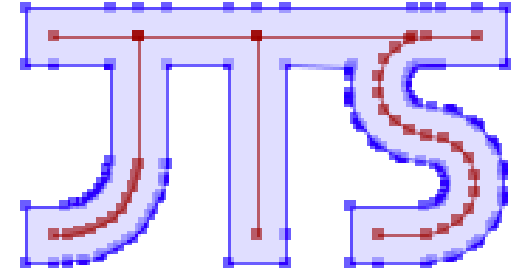
- To get meaningful results (e.g., the length of a line in meters), you need to transform the coordinates from a geographic coordinate system (WGS84) to a projected coordinate system (e.g., UTM, EPSG:32633), where the coordinates represent linear distances, like meters or feet.
- Projected coordinate systems flatten the Earth's surface, allowing for accurate distance and area calculations.
- Use a Spatial Transformation Library: JTS alone does not provide coordinate transformations, but you can integrate it with libraries like Proj4J or GeoTools to perform the coordinate transformation from one system to another.

Step 5: Transformation of Coordinates



- Possible Approach:
 - First, you'll need to define the source coordinate system (WGS84 in this case, EPSG:4326).
 - Then, transform it to a suitable projected coordinate system (e.g., UTM for local distance calculations).
 - Once transformed, you can safely calculate lengths or areas, and the results will be in meaningful units (e.g., meters).

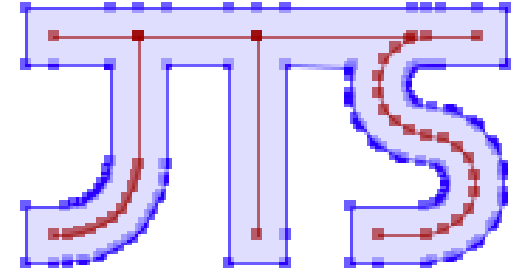
Possible Approach with GeoTools



- Recommended approach (Easier to implement)
- Dependency already in the POM XML file
- Careful: You need the LongitudeFirst flag !!
- (Accounts for the coordinate flip)

```
CoordinateReferenceSystem sourceCRS = null;  
try {  
    sourceCRS = CRS.decode( code: "EPSG:4326", longitudeFirst: true);  
    CoordinateReferenceSystem targetCRS = CRS.decode("EPSG:31256");  
    transform = CRS.findMathTransform(sourceCRS, targetCRS, lenient: true);  
}
```

Step 5: Transformation of Coordinates



- EPSG Codes:
 - EPSG:4326 refers to the WGS84 geographic coordinate system (latitude and longitude in degrees). *(Note: This is just an approximation and only valid in eastern Austria)*
 - EPSG:32633 refers to UTM zone 33N, which is a projected coordinate system where distances are in meters.
- Coordinate Transform:
 - You create a transformation from WGS84 to UTM or any other projected system suitable for your location.
 - Each point in your LineString (or any geometry) must be transformed before calculating lengths or areas.
- Length Calculation After Transformation:
 - After transforming the coordinates to a system where units are linear (like meters), you can safely use `.getLength()` or other geometric methods on your LineString.

Object-oriented Programming 2 KU

OSM, JTS, Geometries, and Data Loading

Sebastian David Albu

Institute of Computer Graphics and Knowledge Visualization (CGV)

Code on slides generated using [Carbon](#)